# Python Functions – Reusable Blocks of Code

**What is a function?**

**A function is a reusable block of code designed to perform a specific task.**

**Example:**

```
def
greet(name):

print("Hello"
, name)
greet("Amit")
greet("Priya"
)
```

**Output:**

```
Hello Amit
Hello Priya
```

Key concepts

- **def** defines a function.
- **greet** is the function name.
- **name** is a parameter that receives input.
- **greet("Amit")** calls the function with an argument.
- A function avoids repeating the same code.

**Real-world analogy: A function is like a machine that performs a specific operation whenever you provide the required input.**



# Parameters, Return Values, and Function Calls

Functions can accept inputs and return results to the caller.

**Example:**

```
def calculate_bonus(salary):
    bonus = salary * 0.10
    return bonus
result = calculate_bonus(50000)
print("Bonus:", result)
```

**Output:**

```
Bonus: 5000.0
```

How it works

1. **calculate_bonus(50000)** calls the function.
2. **salary** receives the value **50000**.
3. The function calculates 10% of the salary.
4. **return** sends the result back to the caller.
5. **result** stores the returned value.

Remember

- **print()** displays information.
- **return** sends a value back for further processing.
- Parameters are defined in the function; arguments are supplied when calling it.

# Python Classes and Objects

**Example:**

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    def display_details(self):
        print("Name:", self.name)
        print("Salary:", self.salary)
emp1 = Employee("Amit", 50000)
emp2 = Employee("Priya", 60000)
emp1.display_details()
emp2.display_details()
```

**What is a class?**

**A class is a blueprint that defines the data and behavior its objects can have.**

**What is an object?**

**An object is an instance created from a class.**

**Output:**

```
Name: Amit
Salary: 50000
Name: Priya
Salary: 60000
```

**Key concepts**

- **`Employee` is the class.**
- **`emp1` and `emp2` are separate objects.**
- **Each object stores its own name and salary.**
- **`__init__()` initializes an object when it is created.**
- **`display_details()` is a method, meaning a function defined inside a class.**

**Real-world analogy: `Employee` is the employee form or blueprint; Amit and Priya are individual employee records created from it.**

# Understanding self and __init__()

What is self?

**self** refers to the current object on which an instance method is operating. It allows a method to access that object's attributes and other methods.

Example:

```
class Employee:
    def __init__(self, name):
        self.name = name
    def greet(self):
        print("Hello", self.name)
emp1 = Employee("Amit")
emp2 = Employee("Priya")
emp1.greet()
emp2.greet()
```

Output:

```
Hello Amit
Hello Priya
```

Remember: **self** connects a method to the particular object whose data it is using.

Understand these statements

- **self.name = name** stores the supplied name in the current object.
- **emp1** stores **"Amit"**; **emp2** stores **"Priya"**.
- **emp1.greet()** automatically passes **emp1** as the first argument to the instance method.
- **self** is a conventional parameter name in Python, not a reserved keyword.

Equivalent concept:

```
emp1.greet()
```

Python effectively invokes the method with **emp1** as its instance argument.

# Functions vs Classes — When to Use Each

Connecting this to our API-processing project

A function-based workflow:

```
fetched_data = fetch_user_data(1)
processed_data = process_data(fetched_data)
export_json(processed_data, "output.json")
```

A class-based workflow:

```
processor = UserDataProcessor()
fetched_data = processor.fetch_user_data(1)
processed_data =
processor.process_data(fetched_data)
processor.export_json(processed_data,
"output.json")
```

Key takeaway: Functions organize tasks. Classes group related data and behavior. Classes can use functions as methods, allowing a larger application to remain organized and reusable.

| Functions | Classes |
|---|---|
| Organize reusable operations | Organize related data and behavior |
| Useful for independent tasks | Useful when objects maintain their own state |
| Called directly, such as **calculate_bonus(50000)** | Methods are usually called through objects, such as **emp1.display_details()** |
| Can accept inputs and return outputs | Can contain attributes, methods, and initialization logic |
| Often ideal for small, focused operations | Often useful for larger workflows with related responsibilities |